An independent reimplementation of the $k = 10$ lonely-runner sieve: $J(10, p)$ is empty for the 61 primes of P_{10}

(a replication, not a new theorem)

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-28

Zenodo DOI to be assigned on release (not peer reviewed)

AI-assistance disclosure and audit status. This preprint, its C and Python code, its Lean code and its claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction; the author is responsible for the content. **This version was independently audited by a different AI agent instance (`AUDIT.md`, ACCEPT_WITH_CORRECTIONS), not by a human.** The audit's six corrections are applied in this text (Section 7). No human audit and no statement lock exist; both are owed (Section 9).

Abstract

Sungkawichai and Trakulthongchai (S–T) reduce the lonely runner conjecture for 11 runners ($k = 10$ moving speeds plus the stationary runner) to a finite computation: for each of 61 primes $p \in P_{10} = \{127, \dots, 467\}$ a set $J(10, p)$ must be empty [1]. We re-ran that computation with *independently written* code. The implementer did not read the S–T repository; the independent auditor opened their logs only after its own counts were complete. **We find $J(10, p) = \emptyset$ for all 61 primes of P_{10}** (exact integer arithmetic, Tier [C]). The step that S–T discharge with their Proposition 1.4 is replaced here by a direct computation: for every p we checked all 2 593 742 461 level-11 vectors (up to scaling) over the class of $(1, \dots, 10)$ and found 0 improper ones. The auditor confirmed this fibre check independently at $p = 127, 137, 263, 467$, and checked S–T Proposition 4.1 exhaustively (15 937 424 600 vectors) and the Lemma 4.3 inclusion at all 61 primes. The scalar inputs are kernel-checked in Lean (Tier [A]): P_{10} is 61 distinct primes, all > 110, equal to row $k = 10$ of S–T's Table 1, and $55^{90} \le 10^{10} \prod P_{10}$, with a margin of 5.08 nats that disappears without the prime 467. **This is a replication, not a new theorem.** Together with the literature premises LRC(9), the Malikiosis–Santos–Schymura/Rosenfeld bound and S–T's Corollary 2.5 and Proposition 2.7 (Tier [L]), the computation gives LRC(10), the conjecture for 11 runners, which is S–T's Theorem 1.3 for $k = 10$. Nothing here is new beyond, to our knowledge, being the first reimplementation of the $k = 10$ computation not derived from the S–T code. The lonely runner conjecture in general is open.

What this is NOT. It is **not** a proof of anything new, **not** a proof of the lonely runner conjecture, and **not** a formalisation of the computation: only scalar inputs are in Lean. The chain to LRC(10) rests on literature premises we did not re-derive (Section 4). "Independent" means independent *code*; the mathematical reduction is S–T's.

Markers. [A]: kernel-checked Lean, axioms within {`propext`, `Classical.choice`, `Quot.sound`}, controls caught. [L]: a cited source, taken as stated. [C]: executed code with an argument written down but not machine-checked. [X]: a recorded absence, supporting nothing by itself. Identifiers `LRI-·-nnnn` refer to `ledger/ledger.json`.

1 Setting and question

Conjecture and notation. LRC(k) denotes the lonely runner conjecture for $k + 1$ runners: k runners with distinct positive integer speeds and one stationary runner, on a circle of circumference 1; for every choice of speeds some time exists at which every moving runner is at distance at least $1/(k + 1)$ from the stationary one. So LRC(9) is the case of 10 runners and LRC(10) the case of 11 runners. Any text calling the present result "ten runners" would be wrong.

The S–T reduction (Tier [L], as read by the producer and re-read by the auditor). Write $\|x\|$ for the distance of x to the nearest integer. For a prime p and $l \geq 1$ let $N = lp$ and let $u \in (\mathbb{Z}_N)^k$ have no coordinate $\equiv 0 \pmod p$. By S–T Definitions 2.1 and 2.3, u is (k, p, l)-*proper* if (gcd) for some index i, $\gcd(l, u_j : j \neq i) > 1$, or (time) some $t \in \frac{1}{lp}\mathbb{Z}$ has $\|tu_i\| \geq \frac{1}{k+1}$ for all i. Let $I(k, p, l)$ be the set of improper tuples. $J(k, p)$ is the set of $v \in \mathbb{Z}_p^{*k}$ lying in the reduction mod p of $I(k, p, l)$ for *every* l. We implement properness exactly as: (gcd) for some prime $q \mid l$ at most one u_j is not divisible by q; or (time) some $a \in [1, N/2]$ has $(k+1)\min(r, N-r) \geq N$ for every $r = au_i \bmod N$. This is the same condition in integers and uses no floating point. The results of S–T used (Tier [L]; `LRI-L-0002`): Lemma 2.4 ($J = \emptyset$ iff $I(k, p, l) = \emptyset$ for some l); Corollary 2.5 (if LRC$(k-1)$ holds and $J(k, p) = \emptyset$ then p divides $u_1 \cdots u_k$ for every counterexample u); Lemma 2.6 (the Malikiosis–Santos–Schymura bound in the form of Rosenfeld's Corollary 3: a counterexample has $\prod u_i < B_k$ with $B_k = \left(\binom{k+1}{2}^{k-1}/k\right)^k$, with division by k, not $k!$); Proposition 2.7 (if LRC$(k-1)$ holds, $J(k, p) = \emptyset$ for all p in a set P and $\prod P \geq B_k$, then LRC(k) holds); and for $k = 10$ the plan $\times 2, \times 2, \times 2, \div 8$ of their Section 5.2.

The question. The preregistration (`data/preregistration.json`, sha256 `140a28ad...`, item LR-IND, class (d): an independent re-run) asked: re-derive $J(10, p) = \emptyset$ for each $p \in P_{10}$ with code written without reading the S–T repository, run positive and negative controls first, and do not call the result a new proof or claim novelty for LRC(10). The answer is yes for all 61 primes. The preregistered text of the item cites Allikvere for the claim that the full computation was not independently reimplemented; that attribution is wrong (Section 7, correction 1) and is not used here as evidence.

2 Method

Producer code is in `data/code/` (C99 `lr_sieve.c`, `fiber_check.c`; Python drivers and checkers; the C sources are also stored as `.c.txt` so that the bundler, which keeps only listed suffixes, includes them). The frozen binaries' sha256 are in `data/logs/freeze_sha256.txt`; compiled binaries are not shipped. The producer wrote the code from the definitions of arXiv 2604.23906 v2 (Defs. 2.1, 2.3, Prop. 3.1, §§5.1–5.2) alone; `data/result.json` records the sha256 of every file before any reading of the S–T or Trakulthongchai repositories. The auditor could not verify that attestation (only the producer can attest it); qualitatively the code is unlike the S–T code (single-file C99 with bitsets, against C++23 templates) and the auditor saw no sign of copying.

Phase 1: classes of $I(k, p, 1)$. [C] One representative per class under permutation, sign and multiplication by $\mathbb{Z}_p^*$. A hitting-set branch: pick a surviving time with the fewest available killers, branch on which speed kills it (excluding siblings already tried), and prune with a capacity bound (the r largest remaining kill counts must cover the surviving times). Supports smaller than k are expanded to all multiplicities at the leaf where they become improper. The branching idea is Rosenfeld's [4]; the two pruning steps were added after a projection of about 230 core-hours for $p = 467$ alone with the plain search (a kill criterion of 40 core-hours was preregistered).

Our own pruning (not from the literature): set-canonical supports. [C] $U = \{1 = u_1 < u_2 < \cdots\} \subseteq [1, (p-1)/2]$ is *set-canonical* if it is lexicographically $\leq \mathrm{fold}(x^{-1}U)$ for every $x \in U$, where $\mathrm{fold}(v) = \min(v, p-v)$. Soundness (re-derived by the auditor): every class has a set-canonical member, since fold merges no two distinct elements of $[1, (p-1)/2]$ that are not $\pm$ each other, so one takes the lexicographically least normalisation; and for distinct $x, y \in U$, $\mathrm{fold}(y/x) \geq u_2$, because the second smallest element of $\mathrm{fold}(x^{-1}U)$ must be $\geq u_2$. Multiplicities travel with the support, so emitting all multiplicity patterns of a set-canonical support is complete. The capacity bound is a valid relaxation.

Lifting and projection. [C] By S–T Proposition 3.1 all c^k preimages of each retained tuple are enumerated and the (k, p, cl)-improper ones are kept; projection reduces mod p and canonicalises. For $k = 10$ the plan is $2, 2, 2, /$ (three lifts by 2, then projection). The soundness of this chain is the invariant of Section 7 (correction 3), not an argument about representatives alone. Each level reports the number of distinct classes of $\pi_p(S)$ and a hash of them; all cross-checks compare at this level, which does not depend on the representatives used.

The level-11 fibre (replacing Proposition 1.4). [C] After the plan $2, 2, 2, /$ the only surviving class is that of $(1, \ldots, 10)$ at every $p \in P_{10}$ (the *identity class*), as S–T's Remark 3.2 predicts. S–T discharge this class by their Proposition 1.4 ($k + 1 = 11$ and $p > k(k + 1)$ prime). We instead computed: `fiber_check 10 p 11` enumerates all $w \in (\mathbb{Z}_{11})^{10}$ up to $\mathbb{Z}_{11}^*$ scaling (the zero vector and the vectors whose first nonzero coordinate is 1: $1 + (11^{10} - 1)/10 = 2\,593\,742\,461$), forms u by the Chinese remainder theorem with $u \equiv (1, \ldots, 10) \pmod{p}$ and $u \equiv w \pmod{11}$, and tests $(10, p, 11)$-properness exactly. Scaling $w \mapsto cw$ ($c \in \mathbb{Z}_{11}^*$) with the matching time is a bijection preserving both tests (audited sound). Total over the 61 primes: 158 218 290 121 fibre vectors, 0 improper. A second program, the generic $\times 11$ lift in `lr_sieve`, enumerated all 25 937 424 601 lifts at $p = 127$ and left 0 (a pre-freeze binary built from the same source as the frozen one).

Optional premise re-run. [C] LRC(9): `lr_sieve 9 p 2,5`, i.e. $I(9, p, 10)$, is empty for all 47 primes of Trakulthongchai's S_9 (137..401), 214 core-seconds. This checks his computation; it is not used as a derivation and LRC(9) remains a literature premise.

3 Results

quantity ($k = 10$, every $p \in P_{10}$, 61 primes)	value
primes with exactly one class (the identity) after $2, 2, 2, /$	61 of 61
classes of $I(10, p, 1)$ ($\lvert S_1 \rvert$): range	1 to 58 087 (max at $p = 137$)
sizes after the lifts, typical	1–4 tuples (581/4/4 at $p = 137$)
level-11 fibre vectors per prime (up to scaling)	2 593 742 461
improper fibre vectors, total over 61 primes	0
generic $\times 11$ lift at $p = 127$: lifts / survivors	25 937 424 601 / 0
unpruned second pass agrees with the main pass (class hashes)	33 of 61 primes (127..307)
core time: main / fibre $\times 11$ / unpruned pass	3 963 s / 5 068 s / 1 326 s
measured time at $p = 127$ / $p = 467$ (final version)	4.7 s / 215 s

So $J(10, p) = \emptyset$ at every $p \in P_{10}$, *without* Proposition 1.4 ([C], exhaustive exact-integer enumeration, `LRI-C-0003`). The Lemma 4.3 inclusion $r_{10}(\frac{1}{11}\mathbb{Z}) \subseteq r_{10}(\frac{1}{p}\mathbb{Z})$ holds exactly for every prime in $[101, 467]$ that was tested (`data/logs/lemma43_check.json`); the audit finds it holds for all 61 primes of P_{10} and, among primes ≤ 113, for exactly $101, 103, 107, 109, 113$, consistent with S–T's footnote 4. Per-prime numbers are in `data/logs/summary.json` and the raw outputs in `data/k10_main/`, `data/k10_fiber11/`, `data/k10_nocanon/`.

3.1 What the auditor recomputed, and what it did not

With its own code and different algorithms (`AUDIT.md`, §2; `LRI-C-0004`):

- Phase 1 and the lifts at $p = 127, 137, 263$: identical class counts and hashes at every level ($S_1 =$ 8 228, 58 087, 17 classes) and only the identity class at the end. At $p = 127$ and 137 it ran *without* the pairwise lemma.
- All 214 positive-control cases $k = 3..8$ with the producer's plans, both with and without the pairwise lemma: 214/214 identical.
- The level-11 fibre with its own program (its own CRT formulation, normalising the *last* nonzero coordinate): 2 593 742 461 leaves and 0 improper at $p = 127, 137, 263, 467$ (also $p = 53, 97$).
- S–T Proposition 4.1 for $k = 10$, exhaustively and without the polynomial method: all 15 937 424 600 vectors of N_{10} ($11^{10} - 10^{10} - 1$) admit $s, r \in \mathbb{Z}_{11}^*$ with $s\,v + r\,(1..10) \in \{1..9\}^{10}$, 0 failures (negative control: with $\{1..8\}$, 186 429 550 failures); the Lemma 4.3 inclusion (exact) at all 61 primes; and a hand re-derivation of the proof of Proposition 4.4. Together these give the identity-fibre conclusion at all 61 primes independently of `fiber_check.c`.
- A no-symmetry brute force of $I(4, 13, 5)$ (12 960 000 ordered tuples, 0 improper), $I(4, 17, 5)$ (0) and the nonempty controls $I(4, 11, 5)$ (23 040) and $I(4, 7, 5)$ (13 824).

Resting on the producer's code at the other primes. Phase 1 and the lifts at the remaining 58 primes (including $p = 467$, where the auditor's own sieve was stopped after 1 898 s with no output, with $\approx$ 6–10 core-hours extrapolated) were not re-run with the auditor's algorithm. They rest on the producer's binaries (rebuilt bit-identically by the auditor from the shipped sources, and rerun at $p = 127, 137, 263, 467$ with identical output) and on the agreement in the next subsection.

3.2 Comparison with the S–T logs

The auditor opened `github.com/vzsky/13-lonely-runners`, `results/result_11` (the $k = 10$ run) only after its own counts at $p = 127$ and 137 were complete (its $p = 263$ run was still running). S–T's "Step 1 ($l = 1$): S size" equals the producer's `S1_classes` at **all** 61 **primes**, with 0 disagreements, and their log reports "Subproof Mod p Done" for all 61. The auditor's own counts agree too. Later sizes are not comparable, since S–T remove the identity class first (their Prop. 1.4) and then lift, whereas we keep it and check the fibre. No disagreement was found, so nothing needs to go to the authors on that account. ([C], `LRI-C-0006`.)

3.3 The $k = 9$ re-run

All 47 primes of S_9 give an empty final set (214 core-seconds). A check of Trakulthongchai's computation only (`LRI-C-0007`); the auditor did not audit it.

4 The chain to LRC(10), with tiers

statement	tier	basis
LRC(9) (the conjecture for 10 runners)	[L]	Trakulthongchai, EJC 33(2) P2.46 [2]
Lemma 2.6: counterexample has $\prod u_i < B_k$, $B_k = (\binom{k+1}{2}^{k-1}/k)^k$, division by k	[L]	Malikiosis–Santos–Schymura [5], Rosenfeld Cor. 3 [4]
Corollary 2.5, Proposition 2.7 (and Lemmas 2.2, 2.4)	[L]	S–T [1]; read, not formalised
P_{10} is 61 distinct primes > 110, equals Table 1 row $k = 10$; 11 prime; $55^{90} \le 10^{10} \prod P_{10}$	[A]	`ScalarInputs.lean`
$J(10, p) = \emptyset$ for all $p \in P_{10}$	[C]	exhaustive enumeration, exact integers (this paper)
Conclusion: LRC(10) (the conjecture for 11 runners)	[L] chain	the rows above; S–T's Theorem 1.3 for $k = 10$; **not new**

The logical chain is: LRC(9), Lemma 2.6, Corollary 2.5 and Proposition 2.7 with $P = P_{10}$, the computation $J(10, p) = \emptyset$ and $\prod P_{10} \ge B_{10}$ give LRC(10). The first three are literature; the last two are this paper. The conclusion is therefore filed at the lowest tier of the chain, [L], and is not a result of this paper. What is filed [C] is the per-prime emptiness; what is filed [A] is the scalar arithmetic, nothing more.

5 Controls

All controls were run before the headline numbers were used (`data/controls_pos/`, `data/controls_neg/`, `data/logs/`; `LRI-C-0005`). Neither Rosenfeld nor Trakulthongchai prints per-prime retained-tuple counts, so the positive controls compare per-prime verdict patterns.

5.1 Positive controls: printed per-prime verdict patterns

The fourth column is renamed from "$J(k, p) = \emptyset$ exactly for" (audit correction 2): what is tested is whether $I(k, p, l_{\text{final}})$ is empty at the level used. A nonempty I at the final level does not by itself imply $J \neq \emptyset$.

k	l (plan)	primes tested	$I(k,p,l_{\text{final}}) = \emptyset$ at the tested level	printed source and match
8	9 (`3,3`)	11..241	47..241 (all 39 primes)	Trakulthongchai [3]: nonempty for $11 \le p \le 43$, empty on $S_8 = \{47..241\}$; exact
7	8 (`2,2,2`)	11..257	every prime ≥ 31 except 41	Rosenfeld [4] §7 ("at least 31 other than 41"); exact (41 leaves 4 classes)
6	7 (`7`)	11..197	$13, 19$ and every prime ≥ 31	Rosenfeld §7: works for $13, 19, \ge 31$; fails for $17, 23, 29$; exact
5	6 (`2,3`)	7..103	every prime ≥ 23	Rosenfeld §7, Table 1; exact
4	5 (`5`)	7..103	13 and every prime ≥ 17	Table 1 $\{17..37\}$ matches; see below
3	4 (`4`)	5..103	every prime ≥ 7	Table 1 $\{7, 11, 13\}$, §7; exact

The $k = 4$, $p = 13$ item (flag withdrawn). The producer had flagged $p = 13$, reading Rosenfeld's remark as predicting failure. Rosenfeld's §7 is hedged: "for $k = 3, 4, 5$ it seems to be true for all prime p larger than 7, 17 and 23 respectively"; his Table 1 uses $\{7, 11, 13\}$, $\{17..37\}$, $\{23..71\}$, and §7 makes no claim that $p = 13$ fails for $k = 4$ (13 is not mentioned). His Lemma 6 condition is $I(k, p, k + 1) = \emptyset$. The auditor's brute force over all 12 960 000 ordered tuples gives $I(4, 13, 5) = \emptyset$, with nonempty controls at $p = 7$ and 11. This agrees with the producer's sieve and with `direct_I.py` (0 improper out of 40 920 multisets). Hence there is no mismatch and `controls_ok` is *true*. (The copy of `result.json` and `RESULT.md` in `data/` is the producer's pre-audit text, in which the flag and `controls_ok: false` remain; the hashes in `result.json` refer to those files. This paper supersedes them on that point.)

Product test and reproductions. The exact test $\prod\{p : J(k,p) = \emptyset,\ p > k+1\} \ge B_k$ holds over the tested ranges for $k = 3..8$. In particular LRC(7) and LRC(8) are reproduced in the S–T Proposition 2.7 form, without Rosenfeld's lcm bonus, assuming LRC(6) and LRC(7) respectively.

5.2 Cross-implementation checks

- `mode_agreement.py`: three phase-1 algorithms (plain hitting set; plus capacity bound; plus canonical-support pruning) on all 56 (k, p) cases from $k = 5$ to 10, including $k = 10$ at $p = 127, 131$ in all three modes: same class counts and hashes at every level, 0 disagreements.
- `brute_py.py`: a naive Python scan of every sorted tuple, no search and no pruning, agrees with `lr_sieve` at every level (count and hash) on 24 cases, $k = 4..7$.
- `direct_vs_sieve.py`: a direct scan of $I(k, p, k + 1)$ over all multisets agrees on emptiness for 14 cases, $k = 3..5$.
- $k = 10$ second pass without the canonical pruning: class hashes equal the main pass at every level for the 33 primes 127..307; stopped there to keep the time box, so $p = 311..467$ were not run unpruned.

5.3 Negative controls

Planted wrong witness rule. Mutation 1 is the preregistered "$<\to\le$" (bad iff $\|x\| \le 1/(k+1)$); mutation 2 uses floor instead of ceiling in the threshold.

case	correct	MUT=1	MUT=2
$k = 7$, $p = 31$	final 0 classes	final **4**	S_1 **0** (vs 19 classes)
$k = 8$, $p = 47$	final 0	final **1**	S_1 120 tuples (vs 1 279)
$k = 8$, $p = 53$	final 0	final **1**	S_1 0 (vs 191)
$k = 10$, $p = 127$	S_1 8 228, final identity	**unchanged**	S_1 311, final **0** (identity wrongly removed)
$k = 10$, $p = 131$	S_1 2 940, final identity	**unchanged**	S_1 86, final 0

Both planted mutations are inert at the level-11 step, and `fiber_check.c` has no mutation switch and was never mutated (audit correction 4). At $k = 10$ the literal "$<\to\leq$" cannot change anything at levels $l \in \{1, 2, 4, 8\}$, because equality $\|x\| = 1/11$ needs $11 \mid lp$; and MUT=2 coincides with the correct rule whenever $11 \mid N$. The mutations do bite where equality is possible ($k = 7$ at $l = 8$, $k = 8$ at $l = 9$), as the table shows. MUT=2 shows the failure mode that matters most: a rounding bug gives a false pass at $k = 10$.

Boundary-sensitive control at the level-11 step (auditor). The strict rule ($\|x\| > 1/11$ in place of $\geq$) at $p = 127$, $L = 11$ flags exactly one improper orbit, $w = (10, 9, \ldots, 1) = -(1..10)$, the tight identity tuple. At $k = 6$, $p = 43$, $L = 7$ the analogous strict control also flags the identity orbit. The correct rule finds 0 at both. Further, the *nonzero* counts of the two independent fibre programs agree: $p = 13$: 54 976 improper orbits in both; $p = 23$: 7 210 in both.

Survivor without the Proposition 1.4 step (preregistered). At all 61 primes the identity class survives $\times 2 \times 2 \times 2 \div 8$ (`identity_survives: true`). `fiber_check` at $L \in \{5, 7\}$ (where $11 \nmid Lp$) finds exactly one improper class, $w = (1..10) \bmod L$, at $p = 127$ and $p = 467$. At $L = 5$, $k = 4$, $p = 7$ (so $p < k(k+1)$) it finds improper tuples: the hypothesis $p > k(k+1)$ matters.

6 Lean (Tier [A], scalar inputs only)

Module `formal_cert/Programmes/LonelyRunnerInd/ScalarInputs.lean` (copy in `data/lean/`), `import Mathlib.Tactic` (never `import Mathlib`), compiled with `scripts/certified_numerics/lean_pinned.py` (Lean 4.34.1, Mathlib `d13f23b7`), rc 0. Theorems: `P10_length` (= 61), `P10_nodup`, `P10_gt_k2k` (every $p > 110$), `P10_prime`, `k_succ_prime`, `choose_11_2`, `B10_le_prod_nat` ($55^{90} \leq 10^{10} \prod P_{10}$, by kernel `decide`), `B10_le_prod` ($((\binom{11}{2})^9/10)^{10} \leq \prod P_{10}$ in $\mathbb{Q}$), and `B10_not_le_prod_without_467`. The margin is $\ln \prod P_{10} - \ln B_{10} = 342.716 - 337.634 = 5.08$ nats, less than $\ln 467 = 6.15$. $B_{10} \approx e^{337.6}$ agrees with S–T Table 1 ("< 338"); with $k!$ in place of k it would be $e^{209.6}$. `#print axioms` (`data/logs/lean_axioms.json`; the auditor's own copy agrees): `P10_length`, `P10_nodup`, `choose_11_2` use none; `P10_gt_k2k`, `B10_le_prod_nat`, `B10_not_le_prod_without_467` use `propext`; `P10_prime`, `k_succ_prime`, `B10_le_prod` use `propext`, `Classical.choice`, `Quot.sound`. The producer's `leanchecker` replay of the module returns rc 0 (`data/logs/leanchecker.json`); the auditor built its own olean and replayed it, rc 0 (control: a nonexistent module gives rc 1). Escape grep (`sorry`, `admit`, `native_decide`, `implemented_by`, `extern`, `axiom`, `unsafe`, `skipKernelTC`, `addDecl`, `ofReduceBool`, and for the auditor also `opaque`, `set_option`, `macro`, `elab`, `debug`): no hits. The producer's Lean runs used `lean_pinned.py`; the auditor compiled a `/tmp` copy with `lean_pinned.py compile` (no `--olean`) and built its own olean for the replay.

Lean controls. `ControlsMustFail.lean` (P_{10} has length 60; the bound without 467) is rejected by the kernel, rc 1; `ControlsSorry.lean` compiles (rc 0) but its axiom audit shows `sorryAx`, which a compile exit code alone would accept (`LRI-A-0002`).

What the Lean says, and only this. P_{10} (typed in from S–T's Table 1; the auditor compared the Lean list with the table text programmatically and with all primes in $[127, 467]$) is a list of 61 distinct primes all > 110; 11 is prime; $\binom{11}{2} = 55$; $55^{90} \leq 10^{10} \prod P_{10}$; dropping 467 breaks the bound. **Not in Lean:** $J(10, p) = \emptyset$, Lemma 2.6, Proposition 2.7, LRC(9), and the link between these numbers and the theorem.

- `ScalarInputs.lean`: `34e9077bd9b1efe8ba477fa16894983d3f26588b7bc28dcac5664e8355e834f7`
- `AxiomAudit.lean`: `245851e5d20afb6bb3fb19558b52874d5bbb82e4dea5e4fe1681e77e50c9b3c7`
- `ControlsMustFail.lean`: `d0730dce355fb9787b83d01cedee7d3c4da384d509619aae8403c1f363deffd0`
- `ControlsSorry.lean`: `af44270912a3a98896cb304654d89d54f89879f741e86748d44760b67c8f883d`

7 Audit corrections applied

The independent audit (`AUDIT.md`, §6) required six corrections to the producer's report. They are applied in this paper as follows.

1. **Attribution.** The producer's report quoted Allikvere (arXiv 2609.02604 v2) as saying that "the full computation has not been independently reimplemented or formally verified". That sentence in Allikvere §6 refers to *his own* $k = 13/14$ computation, not to S–T's $k = 10$. It is not used as evidence. The only novelty statement kept is: to our knowledge (a web and alphaXiv search on 2026-10-28; recency-ranked) this is the first reimplementation of the S–T $k = 10$ computation not derived from the S–T code. Allikvere re-inspected S–T's $k = 12$ residual tuples, not $k = 10$. The searches found no later reimplementation; the newest related items (for example arXiv 2610.23790 on a gap bound, and 2609.06259) are not reimplementations.
2. **Control column and $k = 4$, $p = 13$.** Renamed and withdrawn as in Section 5; `controls_ok` is true.
3. **The soundness of lifting.** The producer's report argued that lifting only representatives is sound, via the identity $\pi_p\big(\pi^{-1}S \cap I(k,p,cl)\big) = \pi_p S \cap \pi_p I(k,p,cl)$. That identity is false for a general S (it holds when S contains every level-l improper tuple over $\pi_p S$); the same identity appears in the proof of S–T Proposition 3.1. We replace it by the invariant actually used. Starting from level-1 representatives r and lifting *full fibres*, the retained set at level l is exactly
$$F_l(r) = \{w \in I(k,p,l) : \ w \equiv r \pmod p\},$$
by Trakulthongchai's Lemma 7 [3] (a tuple improper at cl is improper at l after reduction), as also stated in Allikvere §2 [6] (both read by the auditor, not re-proved here). Hence $r \in J$ implies $F_l(r) \neq \emptyset$ at every level, and $J \subseteq \{\text{classes of } \pi_p(S_n)\}$ at every step. The computation is unaffected, because every plan used chains full fibres from level 1.
4. **Negative controls.** Stated in Section 5: both planted mutations are inert at the level-11 step; `fiber_check.c` was never mutated; the strict-inequality boundary control and the agreement of the nonzero orbit counts (54 976 at $p = 13$; 7 210 at $p = 23$) are cited.
5. **Wording of the Tier [L] consequence.** Stated in Section 4: the chain gives LRC(10), the conjecture for 11 runners, via the premises LRC(9) (10 runners), Lemma 2.6 and S–T Corollary 2.5 / Proposition 2.7; it recomputes S–T's Theorem 1.3 for $k = 10$ and is not a new result.
6. **Python helpers.** They ran under system `python3` without `-I`, reading only files we wrote; harmless. They should be rerun with `-I` if regenerated. (The ledger scripts of this paper run with `-I`.)

An observation on the exposition (not an accusation). The auditor gives a counterexample to the displayed identity in correction 3: take $k = 10$, $p = 127$, $v = (1, \ldots, 10)$ and $S = \{s\}$ where s is the level-2 lift with every coordinate even ($s_i = i$ or $i + 127$). Then s is gcd-proper, so every level-4 lift of s is gcd-proper, hence $S' = \emptyset$ and the left side is empty; but v is improper at level 4 (no time witness since $11 \nmid 4p$; five odd coordinates, so the gcd test fails), so $v \in \pi_p I(10, 127, 4)$ and the right side is $\{v\}$. Here $v \notin J$, so this refutes a proof *step* as displayed, not the proposition's conclusion, and not S–T's computation. This is recorded as an observation about the exposition (also in the proof of S–T Proposition 3.1) and nothing more. It has not been raised with the authors; if anyone raises it, it should go to the authors first, after a second check.

8 Claim ledger

`ledger/build_ledger.py` files 14 claims: 2 at Tier [A] (`LRI-A-0001`, `-0002`), 3 at [L] (`LRI-L-0001..0003`), 8 at [C] (`LRI-C-0001..0008`, the last being the headline) and 1 at [X] (`LRI-X-0001`, recording what was not run). Each carries a content-addressed evidence blob in `ledger/evidence/`; each Tier [A] claim also carries an `ai_audit` field. The gate field `audit` is left null: the audit was done by an AI agent, not a person, so the Elenchus gate keeps `LEDGER_UNAUDITED_TIER_A` open. The gate (`tools/ledger.py ledger.json --evidence-dir evidence`; verbatim in `ledger/gate_output.txt`) exits with code 1: 0 blocking findings and 2 flags, both `LEDGER_UNAUDITED_TIER_A`, one per Tier [A] row. That is the honest state.

9 Limitations

- **Replication, not a new theorem.** The result recomputes S–T's computational input for $k = 10$; LRC(10) (and Theorem 1.3 of S–T for $k = 10$) is theirs. Novelty is only "to our knowledge, first

reimplementation not derived from the S–T code" (a literature and web search of one day, not a priority claim). The lonely runner conjecture in general is open.

- **AI audit only; human audit and statement lock owed.** No person has certified that the Lean statements mean what their gloss says, and no statement lock exists.
- **Unverified, from the audit.** (a) Literature premises (Tier [L]) not re-derived: LRC(9) (EJC 33(2) P2.46) and whether the producer's $k = 9$ re-run matches his S_9 (not audited); the Malikiosis–Santos–Schymura bound and Rosenfeld's Corollary 3 (the AM–GM step was read); the "pre-jump" Lemma 2.2; Lemma 2.4 (read and accepted). (b) **Phase 1 and lifts at** 58 **of the** 61 **primes, including** $p = 467$**,** were not re-run with the auditor's own algorithm (own-algorithm coverage: $127, 137, 263$); they rest on the producer's code (rebuilt bit-identically), its mode agreements and exact S_1 agreement with the S–T log at all 61 primes. (c) The **unpruned** producer pass was finished at 33 of 61 primes only. (d) The identity fibre at all 61 primes is covered independently by Proposition 4.1 (exhaustive) plus the Lemma 4.3 inclusion (exact) plus the proof of Proposition 4.4 (read and re-derived); the auditor's direct level-11 enumeration covers only 4 primes of P_{10}. (e) **Nothing is formalised beyond the scalar inputs.**
- **Blindness is the producer's attestation** and cannot be verified by anyone else; the auditor's reading of the S–T logs came after its own counts at $p = 127, 137$.
- **Deviations from the preregistration.** (1) The preregistration assigns the implementation to a Sonnet-tier agent; the producer was Opus. (2) The canonical-support pruning and capacity bound are the producer's own, checked by mode agreement. (3) The preregistered "$<\to\leq$" control is provably inert at $k = 10$ (run anyway, plus a rounding mutation, and at $k = 7, 8$). (4) The unpruned second pass stopped at 33 of 61 primes. (5) Proposition 1.4 was audited *and* replaced by a computation, beyond the preregistration. (6) Kill criterion: the preregistration asks for throughput at $p = 127$ and a projection; $p = 127$ alone (4.7 s) does not fix the growth rate, so a four-prime fit $(127, 131, 163, 199)$ was used; the first (plain) implementation scaled about p^{10} and projected about 230 core-hours at $p = 467$; the algorithm was changed *before* the full run, checked against the original by mode agreement, and the full run took about 1.1 core-hours (about 3.3 core-hours in all, on two shared cores). (7) The repository guards `antigravity_guard.py` and `test_rigor_guard.py` exit 1 on files elsewhere in the repository, and neither flags a LR-IND file (`data/logs/guard_*.txt`); `pytest tests/` was not run.
- **Pre-freeze files** (kept for the record, not relied on): `logs/control_k8_lift33.jsonl`, `logs/k10_p127_probe.json`, `logs/probe_k10_p163/199/263.json`, `logs/probe_k10_p127_x11.json` (the generic $\times 11$ cross-check at $p = 127$: its binary was built from the same source as the frozen one, before the freeze).
- **Scope.** $k = 10$ only (11 runners). Counts are of classes under the stated symmetries; the S–T-convention tuple counts are given for 33 primes in `summary.json`.
- Not a physical claim of any kind.

10 Reproducibility

Repository AutoevolveAI, branch `worktree-openai-math-discovery`; HEAD when this paper was written: `f2016dc913ac03d301076c1bb8126638a8bec4c1`; the LR-IND outputs and this paper are uncommitted working-tree files at that time. Copies of the inputs are in `data/` (`code/`, `lean/`, `logs/`, `controls_pos/`, `controls_neg/`, `k10_main/`, `k10_fiber11/`, `k10_nocanon/`, `k9_S9/`, `RESULT.md`, `result.json` with the sha256 of every producer file, `preregistration.json`). The command lines below are summarised from the run note and the code headers; they were not re-executed in the paper bundle.

```
gcc -O3 -march=native -o lr_sieve code/lr_sieve.c ; gcc -O3 -march=native -o fiber_check code/fiber_check.c
./lr_sieve 10 127 2,2,2,/        # phase 1, three lifts by 2, projection; prints classes and hashes per level
./fiber_check 10 127 11          # level-11 fibre over (1..10): leaves 2593742461, improper 0
python code/mode_agreement.py ; python code/brute_py.py ; python code/direct_vs_sieve.py
python code/lemma43_check.py
python -I scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/LonelyRunnerInd/ScalarInputs.lean
python -I papers/p5_lonely_runner_ind/ledger/build_ledger.py ; python -I papers/p5_lonely_runner_ind/ledger/run_gate.py
```

References

[1] Sungkawichai, Trakulthongchai, *Eleven, twelve, and thirteen lonely runners*, arXiv:2604.23906 (v2, 2026-09-01). The reduction, Table 1 and the $k = 10$ plan used here ([L]).

[2] Trakulthongchai, proof of the lonely runner conjecture for 10 runners (LRC(9)), Electron. J. Combin. 33(2), P2.46 ([L]; cited as recorded in the producer's notes and the audit).

[3] Trakulthongchai, arXiv:2511.22427; read for its Lemma 7, its $k = 8$ verdict pattern ($S_8 = \{47..241\}$) and its constant C_k; the audit read it for the B_k formula.

[4] Rosenfeld, arXiv:2509.14111; read for §6 (hitting-set search), §7 (verdict patterns for $k = 3..7$), Table 1 and Corollary 3.

[5] Malikiosis, Santos, Schymura, Theorem A (venue as recorded in the producer's notes, not checked), in the form of Rosenfeld's Corollary 3; not re-derived.

[6] Allikvere, *Fourteen and fifteen lonely runners*, arXiv:2609.02604 (v2, 2026-09-24); cited for the B_k formula, §2 (Lemma 7 restated), and as the source of a mis-attribution corrected in Section 7.

[7] The mathlib Community, *The Lean mathematical library*, arXiv:1910.09336 (2019); cited for the library only.